

BÀI 1 TỔNG QUAN VỀ TRÍ TUỆ NHÂN TẠO VÀ KHÔNG GIAN TRẠNG THÁI

I. TỔNG QUAN VỀ TRÍ TUỆ NHÂN TẠO

Trình bày khái niệm về trí tuệ nhân tạo, các thành phần cơ bản trong trí tuệ nhân tạo, đặc trưng của trí tuệ nhân tạo cũng như những nghiên cứu, những ứng dụng của lĩnh vực này.

1.1. KHÁI NIỆM TRÍ TUỆ NHÂN TẠO

Trong khoa học máy tính, **trí tuệ nhân tạo** (*Artificial Intelligence – AI*) (https://vi.wikipedia.org/wiki/Trí_tuệ_nhân_tạo) là trí thông minh được thể hiện bằng máy móc, trái ngược với **trí thông minh tự nhiên** của con người. Thông thường, thuật ngữ “trí tuệ nhân tạo” thường được sử dụng để mô tả các máy móc (hoặc máy tính) có khả năng bắt chước các chức năng “nhận thức” mà con người thường phải liên kết với tâm trí, như “học tập” và “giải quyết vấn đề”.

Trí tuệ nhân tạo (TTNT) là một cách tiếp cận làm cho máy móc suy nghĩ và hành xử một cách thông minh. Những cỗ máy này được điều khiển bởi phần mềm bên trong chúng, vì vậy AI có liên quan nhiều đến các chương trình phần mềm thông minh điều khiển những máy móc này. AI là một ngành khoa học tìm kiếm các lý thuyết và phương pháp luận có thể giúp máy móc hiểu thế giới và phản ứng lại với các tình huống theo cách giống như con người vẫn làm.

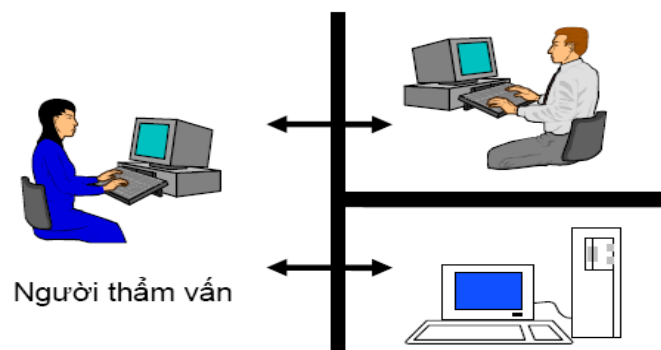
AI có liên quan mật thiết đến việc nghiên cứu bộ não của người. Các nhà nghiên cứu tin rằng AI có thể được thực hiện bằng cách hiểu được cách thức hoạt động của bộ não con người. Bằng cách bắt chước cách bộ não con người làm việc, suy nghĩ và hành động, chúng ta có thể tạo ra một cỗ máy có thể làm được điều tương tự. Cách tiếp cận này có thể được sử dụng như một nền tảng để phát triển các hệ thống thông minh có khả năng học hỏi.

Trí tuệ nhân tạo là một nhánh của khoa học và công nghệ liên quan đến việc làm cho máy tính có những năng lực của trí tuệ con người, tiêu biểu như các khả năng biết suy nghĩ và lập luận để giải quyết vấn đề, biết giao tiếp do hiểu ngôn ngữ và tiếng nói, biết học và tự thích nghi,

Có một số quan điểm khác khi định nghĩa AI: một số nhóm coi trí thông minh là đặc tính của các quá trình suy nghĩ bên trong và lập luận, trong khi những người khác

tập trung vào hành vi thông minh, một đặc điểm bên ngoài. Chúng ta xem xét bốn cách tiếp cận sau đây:

(1) Hành động như con người (Acting humanly): Mong muốn làm cho máy có những khả năng của trí thông minh con người đã có từ nhiều thế kỷ trước, tuy nhiên TTNT chỉ xuất hiện khi con người sang tạo ra máy tính điện tử. Phép thử Turing, được đề xuất bởi Alan Turing (1950), được thiết để trả lời câu hỏi “máy tính có biết nghĩ không?”.



Hình 1.1. Phép thử Turing

Alan Turing đề xuất bộ kiểm thử: Trong phép thử này, một máy tính và một người tham gia trắc nghiệm được đặt vào trong các căn phòng cách biệt với một người thứ hai (người thẩm vấn). Người thẩm vấn không biết được chính xác đối tượng nào là người hay máy tính, và cũng chỉ có thể giao tiếp với hai đối tượng đó thông qua các phương tiện kỹ thuật như một thiết bị soạn thảo văn bản, hay thiết bị đầu cuối. Người thẩm vấn có nhiệm vụ phân biệt người với máy tính bằng cách chỉ dựa trên những câu trả lời của họ đối với những câu hỏi được truyền qua thiết bị liên lạc này. Trong trường hợp nếu người thẩm vấn không thể phân biệt được máy tính với người thì khi đó theo Turing máy tính này có thể được xem là thông minh. Để vượt qua phép thử Turing, máy tính cần có những khả năng sau:

- Xử lý ngôn ngữ tự nhiên để giao tiếp thành công bằng ngôn ngữ của con người;
- Biểu diễn tri thức để lưu trữ những gì hệ thống biết được hoặc nghe được;
- Suy luận tự động để trả lời câu hỏi và đưa ra kết luận mới;
- Học máy để thích ứng với hoàn cảnh mới.

(2) Nghĩ như con người (Thinking humanly): Phương pháp tiếp cận mô hình nhận thức. Để nói rằng một chương trình suy nghĩ giống như con người, ta phải biết con người suy nghĩ như thế nào. Chúng ta có thể tìm hiểu về cách nghĩ của con người theo các cách sau:

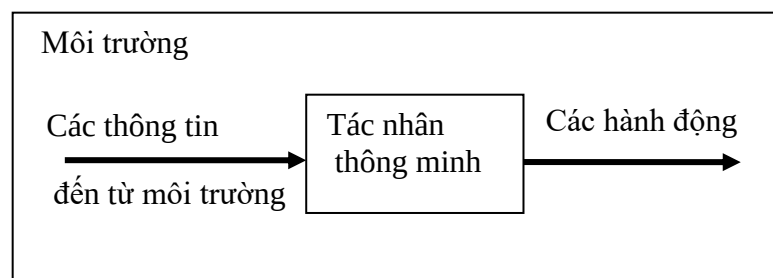
Nội tâm - cố gắng nắm bắt những suy nghĩ của chính chúng ta khi chúng trôi qua;
Thực nghiệm tâm lý - quan sát một người đang hành động;
Hình ảnh não - quan sát não hoạt động.

Một khi chúng ta có một lý thuyết đủ chính xác về trí não, thì có thể diễn đạt lý thuyết như một chương trình máy tính. Nếu hành vi đầu vào - đầu ra của chương trình phù hợp với hành vi tương ứng của con người, đó là bằng chứng cho thấy một số cơ chế của chương trình cũng có thể hoạt động giống như ở con người.

(3) Suy nghĩ hợp lý (Thinking rationally): Cách tiếp cận “quy luật của suy nghĩ”.

Nhà triết học Hy Lạp Aristotle là một trong những người đầu tiên cố gắng hệ thống hóa “tư duy đúng đắn” - nghĩa là các quá trình suy luận không thể bác bỏ. Tam đoạn luận của ông đã cung cấp các mẫu cho cấu trúc lập luận luôn mang lại kết luận đúng khi các tiền đề cho trước đúng.

(4) Hành động hợp lý (Acting rationally): Cách tiếp cận tác nhân hợp lý. Tác nhân hợp lý là tác nhân hoạt động để đạt được kết quả tốt nhất hoặc kết quả mong đợi tốt nhất khi có yếu tố không chắc chắn.



Hình 1.2. Mô hình tác nhân thông minh

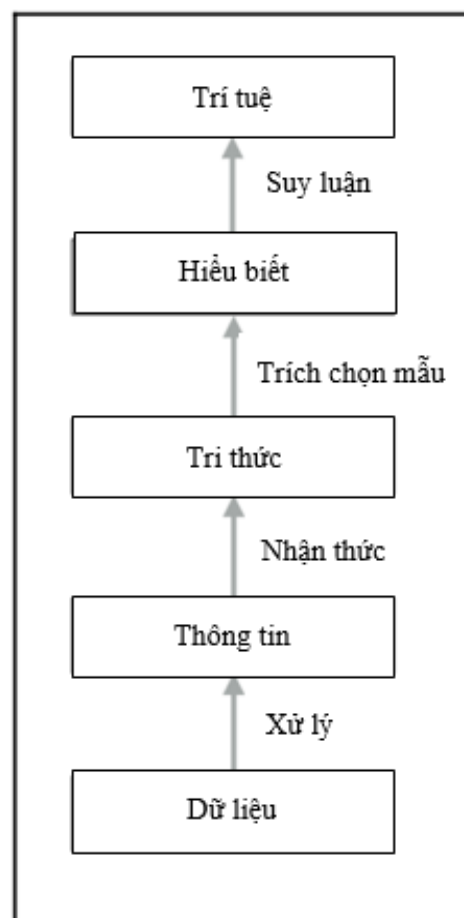
1.2. TẠI SAO CẦN TRÍ TUỆ NHÂN TẠO

AI có khả năng tác động đến mọi khía cạnh trong cuộc sống của chúng ta. Với AI, chúng ta muốn xây dựng các hệ thống thông minh cũng như hiểu khái niệm trí thông minh. Các hệ thống thông minh mà chúng ta xây dựng rất hữu ích trong việc

hiểu cách một hệ thống thông minh vận hành giống như bộ não của chúng ta tiến hành xây dựng một hệ thống thông minh khác.

So với một số lĩnh vực khác như Toán học hay Vật lý đã có từ nhiều thế kỷ trước, AI còn tương đối sơ khai. Trong vài thập kỷ qua, AI đã tạo ra một số sản phẩm ngoạn mục như ô tô tự lái và robot thông minh có thể đi bộ. Dựa trên định hướng mà chúng ta đang hướng tới, rõ ràng là việc đạt được trí thông minh sẽ có tác động lớn đến cuộc sống của chúng ta trong những năm tới.

Chúng ta luôn tự hỏi làm thế nào mà bộ não con người có thể làm được nhiều việc một cách dễ dàng như vậy. Chúng ta có thể dễ dàng nhận ra các đồ vật, hiểu ngôn ngữ, học những điều mới và thực hiện nhiều nhiệm vụ phức tạp hơn với bộ não của mình. Làm thế nào để bộ não con người làm được điều này? Khi ta cố gắng làm điều này với một chiếc máy, ta sẽ thấy rằng nó bị tụt lại phía sau! Tất cả những gì chúng ta phải làm là bắt chước chức năng của của bộ não con người để tạo ra một hệ thống thông minh có thể làm điều gì đó tương tự, thậm chí có thể làm tốt hơn.



Hình 1.3. Quá trình hình thành trí tuệ của con người

Một trong những lý do chính mà chúng ta muốn nghiên cứu về AI là để thực hiện tự động hóa nhiều thứ. Chúng ta đang sống trong một thế giới nơi:

- Hằng ngày chúng ta xử lý lượng dữ liệu khổng lồ và không thể vượt qua. Bộ não con người không thể theo dõi quá nhiều dữ liệu.
- Dữ liệu đến đồng thời và từ nhiều nguồn từ khác nhau.
- Dữ liệu không có tổ chức và hỗn loạn.
- Tri thức thu được từ dữ liệu này phải được cập nhật liên tục vì bản thân dữ liệu luôn thay đổi.
- Thu nhận và hành động phải diễn ra trong thời gian thực với độ chính xác cao.

Mặc dù bộ não của con người rất giỏi trong việc phân tích mọi thứ xung quanh chúng ta, nhưng nó không thể theo kịp các điều kiện trước đó. Do đó, chúng ta cần thiết kế và phát triển những cỗ máy thông minh để có thể làm được điều này. Chúng ta mong muốn các hệ thống AI có thể:

- Xử lý một lượng lớn dữ liệu một cách hiệu quả. Với sự ra đời của Điện toán đám mây, giờ đây chúng ta có thể lưu trữ một lượng lớn dữ liệu.
- Nhập dữ liệu đồng thời từ nhiều nguồn mà không có bất kỳ độ trễ nào.
- Lập chỉ mục và tổ chức dữ liệu theo cách cho phép chúng tôi thu thập thông tin chi tiết.
- Học hỏi từ dữ liệu mới và cập nhật liên tục bằng cách sử dụng các thuật toán học phù hợp.
- Suy nghĩ và phản ứng với các tình huống dựa trên các điều kiện trong thời gian thực.

Các kỹ thuật AI đang tích cực được sử dụng để làm cho các máy hiện có thông minh hơn, để chúng có thể thực thi nhanh hơn và hiệu quả hơn.

1.3. ỨNG DỤNG CỦA AI

AI đã được sử dụng trong nhiều ngành công nghiệp và nó tiếp tục mở rộng nhanh chóng. Một số lĩnh vực phổ biến nhất bao gồm:

Thị giác máy tính (Computer Vision): nghiên cứu về việc thu nhận, xử lý, nhận dạng thông tin hình ảnh thành biểu diễn mức cao hơn như các đối tượng xung quanh để máy tính có thể hiểu được. Đây là những hệ thống xử lý dữ liệu trực quan như hình ảnh và video. Các hệ thống này hiểu nội dung và trích xuất thông tin chi tiết dựa trên trường hợp sử dụng. Ví dụ: Google sử dụng tìm kiếm hình ảnh đảo ngược để tìm kiếm các hình ảnh tương tự về mặt hình ảnh trên Web (Hình 1.4)

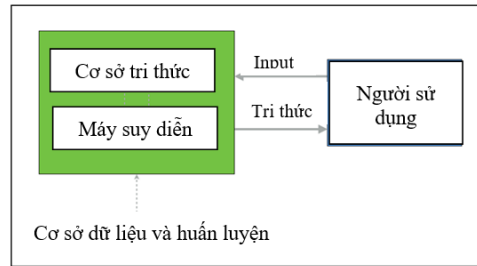
Xử lý ngôn ngữ tự nhiên (Natural Language Processing): Lĩnh vực này đề cập đến việc hiểu văn bản. Chúng ta có thể tương tác với máy bằng cách gõ các câu ngôn ngữ tự nhiên. Các công cụ tìm kiếm sử dụng thông tin này để cung cấp các kết quả tìm kiếm phù hợp.

Nhận dạng giọng nói (Speech Recognition): Các hệ thống này có khả năng nghe và hiểu các từ được nói. Ví dụ: có những trợ lý cá nhân thông minh trên điện thoại thông minh của chúng ta có thể hiểu những gì chúng ta đang nói và cung cấp thông tin liên quan hoặc thực hiện một hành động dựa trên đó.



Hình 1.4. Kết quả tìm kiếm với hình ảnh thay đổi của Google trên Web

Hệ chuyên gia (Expert Systems): Các hệ chuyên gia này sử dụng các kỹ thuật AI để đưa ra lời khuyên hoặc đưa ra quyết định. Các hệ này thường sử dụng cơ sở dữ liệu về các lĩnh vực kiến thức chuyên ngành như tài chính, y học, tiếp thị, v.v. để đưa ra lời khuyên về những việc cần làm tiếp theo. Hãy xem một hệ thống chuyên gia trông như thế nào và nó tương tác với người dùng như thế nào:



Hình 1.4. Hoạt động của Hệ chuyên gia

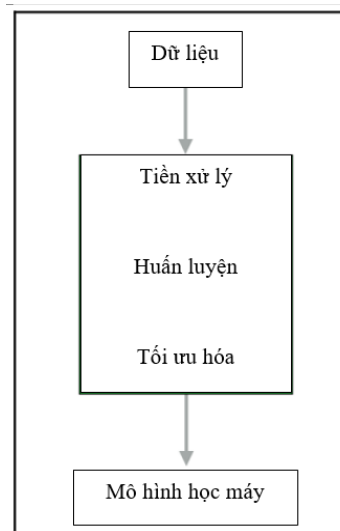
Trò chơi (Games): AI được sử dụng rộng rãi trong ngành công nghiệp trò chơi. Nó được sử dụng để thiết kế các tác nhân thông minh có thể cạnh tranh với con người. Ví dụ, AlphaGo là chương trình máy tính có thể chơi trò chơi chiến thuật cờ vây. AI cũng được sử dụng để thiết kế nhiều loại trò chơi khác nơi mà chúng ta mong đợi máy tính hoạt động một cách thông minh.

Người máy (Robotics): người máy kết hợp nhiều khái niệm trong AI. Các hệ thống này có thể thực hiện nhiều nhiệm vụ khác nhau. Tùy thuộc vào từng tình huống cụ thể, người máy có cảm biến và cơ cấu truyền động có thể làm những việc khác nhau. Các cảm biến này có thể nhìn thấy mọi thứ ở phía trước và đo nhiệt độ, độ nóng, chuyển động, v.v. Chúng có bộ xử lý trên bo mạch để thực hiện nhiều nhiệm vụ khác nhau trong thời gian thực. Chúng cũng có khả năng thích nghi với môi trường mới.

1.4. CÁC LĨNH VỰC NGHIÊN CỨU CỦA AI

Có nhiều kỹ thuật nghiên cứu, phát triển ngành khoa học AI, dưới đây là các chủ đề đang được các nhà nghiên cứu quan tâm:

Học máy và nhận dạng mẫu (Machine learning and pattern recognition): Chủ đề này có lẽ đang được nhiều người quan tâm nhất hiện nay. Chúng ta thiết kế và phát triển phần mềm có thể học hỏi từ dữ liệu. Dựa trên các mô hình học này, chúng ta thực hiện các dự đoán trên dữ liệu chưa biết. Một trong những hạn chế chính ở đây là các chương trình này bị giới hạn sức mạnh của dữ liệu. Nếu tập dữ liệu nhỏ, thì các mô hình học tập cũng sẽ bị hạn chế. Dưới đây là một hệ thống học máy điển hình:



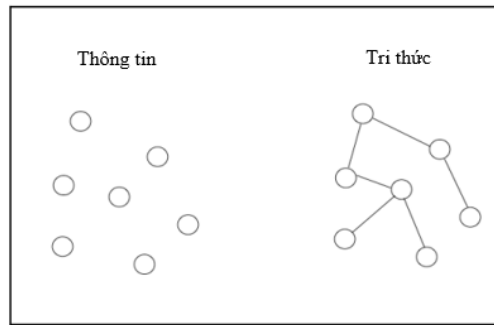
Hình 1.4. Mô hình học máy

Khi hệ thống thực hiện một quan sát, nó được huấn luyện để so sánh quan sát này với mẫu đã có trước đó. Ví dụ, trong hệ thống nhận dạng khuôn mặt, phần mềm sẽ cố gắng đối sánh mẫu mắt, mũi, môi, lông mày, v.v. để tìm khuôn mặt trong cơ sở dữ liệu hiện có của người dùng.

AI dựa trên logic (Logic-based AI): Logic toán học được sử dụng để thực thi các chương trình máy tính trong AI dựa trên logic. Một chương trình được viết bằng AI dựa trên logic về cơ bản là một tập hợp các câu lệnh ở dạng logic thể hiện các *sự kiện* và *luật* về một bài toán cụ thể nào đó. Công cụ này được sử dụng rộng rãi trong đối sánh mẫu, phân tích cú pháp ngôn ngữ, phân tích ngữ nghĩa, v.v.

Tìm kiếm (Search): Các kỹ thuật tìm kiếm được sử dụng rộng rãi trong các chương trình AI. Các chương trình này kiểm tra một số lượng lớn các khả năng và sau đó chọn ra phương án tối ưu nhất. Ví dụ, kỹ thuật này được sử dụng nhiều trong các trò chơi chiến lược như Cờ vua, phân bổ tài nguyên, lập kế hoạch, v.v.

Biểu diễn tri thức (Knowledge representation): Các sự kiện về thế giới xung quanh chúng ta cần được biểu diễn theo một cách nào đó để một hệ thống hiểu được chúng. Các ngôn ngữ logic toán học thường được sử dụng trong trường hợp này. Nếu tri thức được thể biểu diễn một cách hiệu quả, thì hệ thống trở nên thể thông minh. Ontology là một lĩnh vực nghiên cứu liên quan chặt chẽ đến các loại đối tượng đang tồn tại. Nó có thể định nghĩa một cách hình thức về các thuộc tính và mối quan hệ của các thực thể tồn tại trong một miền cụ thể nào đó. Sơ đồ sau đây cho thấy sự khác biệt giữa thông tin và tri thức:



Hình 1.5. Sự khác biệt giữa thông tin và tri thức

Lập kế hoạch (Planning): Lĩnh vực này đề cập đến việc lập kế hoạch tối ưu mang lại cho chúng ta lợi nhuận tối đa với chi phí tối thiểu. Các chương trình này bắt đầu với các dữ kiện về tình huống cụ thể và mục tiêu cần đạt được. Các chương trình này cũng nhận thức được các quy luật xuất phát từ các sự kiện của môi trường xung quanh. Từ những thông tin này, chúng đưa ra phương án tối ưu nhất để đạt được mục tiêu.

Lý thuyết tìm kiếm heuristics: Heuristic là một kỹ thuật hữu ích trong việc giải quyết vấn đề trong ngắn hạn, nhưng không được đảm bảo được tính tối ưu. Kỹ thuật này bao gồm các phương pháp và các kỹ thuật tìm kiếm, sử dụng các tri thức đặc biệt nảy sinh từ bản thân bài toán cần giải để từ đó nhanh chóng đưa ra kết quả mong muốn. Trong AI, chúng ta thường xuyên gặp phải những tình huống mà chúng ta không thể kiểm tra mọi khả năng để chọn ra phương án tốt nhất. Vì vậy, chúng ta cần sử dụng heuristics để đạt được mục tiêu. Chúng được sử dụng rộng rãi trong các lĩnh vực như robot, công cụ tìm kiếm, v.v.

Lý thuyết tìm kiếm heuristics: bao gồm các phương pháp và các kỹ thuật tìm kiếm, sử dụng các tri thức đặc biệt nảy sinh từ bản thân lĩnh vực của bài toán cần giải để từ đó nhanh chóng đưa ra kết quả mong muốn

Lập trình di truyền (Genetic programming): Lập trình di truyền là một phương pháp để có được các chương trình để giải quyết một nhiệm vụ, bằng cách ghép đôi các chương trình và chọn chương trình phù hợp nhất. Các chương trình được mã hóa như một tập hợp các gen, sử dụng một thuật toán để có được một chương trình có khả năng thực hiện các nhiệm vụ thực sự tốt.

1.4. LỊCH SỬ HÌNH THÀNH VÀ PHÁT TRIỂN AI

Một cách nhanh chóng để tóm tắt các cột mốc quan trọng trong lịch sử AI là liệt kê những người giành Giải thưởng Turing: Marvin Minsky (1969) và John McCarthy

(1971) vì đã xác định nền tảng của lĩnh vực này dựa trên biểu diễn tri thức và lập luận tri; Ed Feigenbaum và Raj Reddy (1994) để phát triển các hệ chuyên gia mã hóa tri thức của con người để giải quyết các vấn đề trong thế giới thực; Judea Pearl (2011) phát triển các kỹ thuật lập luận dựa trên xác suất để đối phó với sự không chắc chắn một cách có nguyên tắc; và cuối cùng là Yoshua Bengio, Geoffrey Hinton và Yann LeCun (2019) vì đã biến “học sâu” (mạng nơ-ron đa lớp) trở thành một phần quan trọng của máy tính hiện đại. Phần còn lại của phần này đi sâu hơn vào từng giai đoạn của lịch sử AI.

Sự ra đời của trí tuệ nhân tạo (1943 – 1956)

- Công trình đầu tiên mà ngày nay thường được công nhận là AI được thực hiện bởi Warren McCulloch và Walter Pitts (1943), họ đã đề xuất một mô hình nơ-ron nhân tạo. McCulloch và Pitts cũng gợi ý rằng các mạng được xác định phù hợp có khả năng học hỏi.
- Donald Hebb (1949) đã chứng minh một quy tắc cập nhật đơn giản để điều chỉnh trọng số kết nối giữa các nơ-ron. Quy tắc của ông, bây giờ được gọi là quy tắc Hebbian, vẫn là một mô hình có ảnh hưởng cho đến ngày nay.
- Năm 1955, John McCarthy của Đại học Dartmouth đã thuyết phục Minsky, Claude Shannon và Nathaniel Rochester giúp ông tập hợp các nhà nghiên cứu Hoa Kỳ quan tâm đến lý thuyết automata, mạng nơ-ron nhân tạo và nghiên cứu trí thông minh. Họ tổ chức một hội thảo kéo dài hai tháng tại Dartmouth vào mùa hè năm 1956. Chính vì lý do này mà người ta lấy hội nghị này làm sự kiện ra đời của ngành AI.

Sự nhiệt tình ban đầu, những kỳ vọng lớn (1952 – 1969)

Tiếp nối sự thành công chương trình LT (Logic Theorist), Newell và Simon đã đề xuất mô hình GPS (General Problem Solver). GPS có lẽ là chương trình đầu tiên thể hiện phương pháp tiếp cận “suy nghĩ theo con người”. Sự thành công của GPS và các chương trình tiếp theo làm mô hình nhận thức đã khiến Newell và Simon (1976) hình thành giả thuyết hệ thống ký hiệu vật lý nổi tiếng, trong đó tuyên bố rằng “một hệ thống ký hiệu vật lý là phương tiện cần và đủ cho hành động thông minh nói chung.” Ý của họ là bất kỳ hệ thống nào (con người hoặc máy móc) thể hiện trí thông minh đều phải hoạt động bằng cách điều khiển cấu trúc dữ liệu bao gồm các ký hiệu.

Năm 1958, John McCarthy đã có hai đóng góp quan trọng cho AI. Trong MIT AI Lab Memo No.1, ông đã định nghĩa ngôn ngữ cấp cao Lisp, ngôn ngữ này đã trở thành

ngôn ngữ lập trình AI thống trị trong 30 năm. Trong một bài báo có tựa đề “Programs with Common Sense” ông đã nâng tầm đề xuất khái niệm cho các hệ thống AI dựa trên tri thức và lập luận. Cách tiếp cận này vẫn còn phù hợp cho đến ngày nay.

Năm 1963, McCarthy bắt đầu làm việc phòng thí nghiệm AI tại Stanford. Ông lên kế hoạch xây dựng chương trình lời khuyên cuối cùng Advice Taker. Chương trình này của ông đã được nâng cao bởi J.A.Robinson vào năm 1965 về phương pháp hợp giải (một thuật toán chứng minh định lý hoàn chỉnh cho logic vị từ bậc nhất).

Chương trình ANALOGY (1968) của Tom Evans đã giải được các bài toán hình học xuất hiện trong các bài kiểm tra IQ.

Trong giai đoạn này, mạng nơ-ron của McCulloch và Pitts được các nhà nghiên cứu quan tâm phát triển mạnh mẽ. Phương pháp học của Hebb được nâng cao bởi Bernie Widrow (Widrow và Hoff, 1960; Widrow, 1962), người đã gọi các mạng của mình là ADALINES và Frank Rosenblatt (1962) bằng các Perceptron của mình. Định lý hội tụ Perceptron (Block và cộng sự, 1962) nói rằng thuật toán học có thể điều chỉnh trọng số kết nối của Perceptron có thể phù hợp với bất kỳ dữ liệu đầu vào nào, miễn là tồn tại sự phù hợp như vậy.

Sự trầm lắng (1966 – 1973)

Ngay từ đầu, các nhà nghiên cứu AI đã không ngại đưa ra dự đoán về những thành công sắp tới của họ. Simon đưa ra dự đoán cụ thể hơn: trong vòng 10 năm nữa máy tính sẽ trở thành nhà vô địch cờ vua và một định lý toán học quan trọng sẽ được chứng minh bằng máy. Những dự đoán này đã trở thành sự thật (hoặc gần đúng) trong vòng 40 năm thay vì 10 năm. Sự tự tin quá mức của Simon là do hiệu suất đầy hứa hẹn của các hệ thống AI ban đầu trên các ví dụ đơn giản. Tuy nhiên, trong hầu hết mọi trường hợp, những hệ thống ban đầu này đều thất bại với những bài toán khó khăn hơn.

Có hai lý do chính cho sự thất bại này. Thứ nhất, là nhiều hệ thống AI ban đầu chủ yếu dựa trên "sự xem xét nội tâm được thông báo" về cách con người thực hiện một nhiệm vụ, thay vì phân tích kỹ lưỡng nhiệm vụ, ý nghĩa của một giải pháp và thuật toán cần phải làm gì để tạo ra các giải pháp như vậy một cách đáng tin cậy. Thứ hai là sự thiếu đánh giá cao về khả năng khó giải quyết của nhiều vấn đề mà AI đang cố gắng giải quyết. Hầu hết các hệ thống giải quyết vấn đề ban đầu đều hoạt động bằng cách thử các bước kết hợp khác nhau cho đến khi tìm ra giải pháp. Chiến lược này ban đầu hoạt động vì *thế giới nhỏ* (microworlds) chứa rất ít đối tượng và do đó rất ít hành động có thể thực hiện được và chuỗi giải pháp rất ngắn.

Hệ chuyên gia (1969–1986)

Bức tranh về giải quyết vấn đề đã nảy sinh trong thập kỷ đầu tiên của nghiên cứu AI là một cơ chế tìm kiếm có mục đích chung cố gắng xâu chuỗi các bước lập luận cơ bản lại với nhau để tìm ra các giải pháp hoàn chỉnh. Cách tiếp cận này có một số hạn chế do không có tri thức về lĩnh vực liên quan, và do vậy không thể giải quyết những bài toán khó, đòi hỏi khối lượng tính toán lớn hoặc nhiều tri thức chuyên sâu. Giải pháp thay thế cho các phương pháp này là sử dụng tri thức mạnh hơn, theo lĩnh vực cụ thể cho phép các bước lập luận lớn hơn và có thể dễ dàng xử lý các trường hợp thường xảy ra trong các lĩnh vực chuyên môn hẹp hơn. Có thể kể đến một số chương trình như vậy:

- Chương trình DENDRAL (Buchanan và cộng sự, 1969) là một minh chứng ban đầu của cách tiếp cận này. Nó được phát triển tại Stanford, nơi Ed Feigenbaum (một học trò cũ của Herbert Simon), Bruce Buchanan (một triết gia trở thành nhà khoa học máy tính), và Joshua Lederberg (một nhà di truyền học đoạt giải Nobel) đã hợp tác để giải quyết vấn đề dự đoán cấu trúc phân tử từ thông tin được cung cấp bởi một khối phổ kế.
- Nỗ lực lớn tiếp theo là hệ thống MYCIN để chẩn đoán nhiễm trùng máu. Với khoảng 450 quy tắc, MYCIN có thể hoạt động tốt như một số chuyên gia và tốt hơn đáng kể so với các bác sĩ sơ cấp.

Năm 1972, Alain Colmerauer đã phát triển ngôn ngữ Prolog (Programming in logic - tức là lập trình logic) phục vụ việc biểu diễn tri thức dưới dạng tương tự logic vị từ và lập luận trên tri thức đó.

Sự trở lại của mạng nơ-ron (1986 – nay)

Vào giữa những năm 1980, ít nhất bốn nhóm khác nhau đã phát minh lại thuật toán học truyền ngược (back-propagation learning algorithm) được phát triển lần đầu tiên vào đầu những năm 1960. Thuật toán đã được áp dụng cho nhiều bài toán trong khoa học máy tính và tâm lý học, và việc phổ biến rộng rãi các kết quả trong “Parallel Distributed Processing” (Rumelhart và McClelland, 1986) đã tạo ra rất nhiều hứng thú cho các nhà nghiên cứu.

Lập luận xác suất và học máy (1987 – nay)

Tính thiếu ổn định của các hệ chuyên gia đã dẫn đến một cách tiếp cận mới, khoa học hơn đó là dựa trên xác suất thay vì logic hai trị, học máy thay vì mã hóa bằng tay, và các kết quả thực nghiệm hơn là các tuyên bố triết học. Cách tiếp cận này dựa trên các định lý được chứng minh một cách chặt chẽ hoặc phương pháp thực nghiệm vững chắc (Cohen, 1995) hơn là dựa trên trực giác. Chẳng hạn, trong những năm 1980, mô

hình Markov ẩn (Hidden Markov Models - HMM) đã thống trị lĩnh vực nhận dạng giọng nói, hay Pearl (1988) đã phát triển mạng Bayes, đây là công cụ hiệu quả mạnh mẽ để lập luận với các tri thức không chắc chắn.

Dữ liệu lớn (2001 – nay)

Những tiến bộ đáng kể trong sức mạnh tính toán và sự ra đời của World Wide Web đã tạo điều kiện thuận lợi cho việc tạo ra các tập dữ liệu rất lớn (đôi khi được gọi là dữ liệu lớn – Big data). Các tập dữ liệu này bao gồm hàng nghìn tỷ từ văn bản, hàng tỷ hình ảnh, hàng tỷ giờ nói và video, cũng như một lượng lớn dữ liệu bộ gen, dữ liệu theo dõi xe, dữ liệu dòng nhấp chuột, dữ liệu mạng xã hội, v.v. Điều này đã dẫn đến sự phát triển của các thuật toán học được thiết kế đặc biệt để tận dụng các tập dữ liệu rất lớn này. Có thể kể đến các thuật toán của Banko and Brill (2001), Hays and Efros (2007), (Deng và các cộng sự, 2009), Havenstein, (2005), Halevy và các cộng sự (2009).

Dữ liệu lớn là một yếu tố quan trọng trong chiến thắng năm 2011 của hệ thống Watson thuộc hãng IBM trước các nhà vô địch con người trong cuộc thi Jeopardy (trò chơi đố vui), một sự kiện có tác động lớn đến nhận thức của công chúng về AI.

Học sâu (2011 – nay)

Thuật ngữ học sâu (Deep learning) đề cập đến việc học máy sử dụng mạng nhiều lớp nơ-ron đơn, có thể điều chỉnh. Các thí nghiệm đã được thực hiện với các mạng như vậy từ những năm 1970, và ở dạng mạng nơ-ron tích chập (Convolutional Neural Network – CNN), các nhà nghiên cứu đã tìm thấy một số thành công trong việc nhận dạng chữ số viết tay vào những năm 1990 (LeCun và cộng sự, 1995). Tuy nhiên, phải đến năm 2011, phương pháp học sâu mới thực sự thành công. Thành công đầu tiên cho bài toán nhận dạng giọng nói và sau đó là nhận dạng đối tượng trực quan.

Trong cuộc thi ImageNet năm 2012, người tham dự yêu cầu phân loại hình ảnh thành một trong một nghìn danh mục (cánh tay, giá sách, nút vặn, v.v. Geoffrey Hinton tại Đại học Toronto (Krizhevsky và cộng sự, 2013) đã tạo ra mạng học sâu, kết quả thu được là cải tiến đáng kể so với các hệ thống trước đó. Kể từ đó, các hệ thống học sâu đã vượt quá hiệu suất của con người trong một số nhiệm vụ liên quan đến thị giác. Lợi ích tương tự cũng đã được báo cáo trong việc nhận dạng giọng nói, dịch máy, chẩn đoán y tế và chơi trò chơi. Việc sử dụng mạng học sâu để thể hiện chức năng đánh giá đã góp phần vào chiến thắng của ALPHAGO trước những kỳ thủ cờ vây hàng đầu (Silver và các cộng sự, 2016, 2017, 2018).

Học sâu chủ yếu dựa vào phần cứng mạnh mẽ. Trong khi một CPU máy tính tiêu chuẩn có thể thực hiện 10^9 hoặc 10^{10} thao tác trên giây. Một thuật toán học sâu chạy trên phần cứng chuyên dụng (ví dụ: GPU, TPU hoặc FPGA) có thể thực hiện 10^{14} đến 10^{17} phép toán trên giây, chủ yếu ở dạng ma trận được song song cao và các phép toán vector.

Tổng kết

- ❖ Như vậy TTNT là là một lĩnh vực của khoa học và công nghệ nhằm làm cho máy có những khả năng của trí tuệ con người, tiêu biểu như biết suy nghĩ và lập luận để giải quyết vấn đề, biết giao tiếp do hiểu ngôn ngữ tự nhiên và tiếng nói, biết học và tự thích nghi, v.v.
 - ❖ Sự phát triển của TTNT đã tạo ra một bước nhảy vọt về chất trong kỹ thuật và kỹ nghệ xử lý thông tin. Trí tuệ nhân tạo chính là cơ sở của công nghệ xử lý thông tin mới.
 - ❖ TTNT có vai trò rất quan trọng trong việc đưa ra lời giải cho các bài toán có không gian tìm kiếm lớn.
 - ❖ TTNT gồm hai thành phần cơ bản: tri thức và suy diễn
- $$AI = \text{Tri thức} + \text{Suy diễn}$$
- ❖ TTNT được ứng dụng thành công trong hầu hết các lĩnh vực: Kinh tế, địa chất, y học, hóa học, v.v. với mô hình học sâu hiện nay.

Bài tập

Bài 1

- a) Trí tuệ nhân tạo là gì? Cho ví dụ một chương trình sử dụng TTNT
- b) Cho biết vai trò của TTNT và cho ví dụ minh họa cho nhận định này
- c) Các thành phần cơ bản của TTNT

Bài 2: Nghiên cứu tài liệu AI để tìm ra công việc nào dưới đây có thể giải quyết được bằng máy tính

- a) Trò chơi bóng bàn
- b) Đưa ra lời khuyên, tư vấn về một căn bệnh nào đó
- c) Viết một truyện cười
- d) Dịch tiếng Anh sang tiếng Việt theo thời gian thực

II. KHÔNG GIAN TRẠNG THÁI

Giới thiệu cách biểu diễn bài toán trong không gian trạng thái, từ không gian này chuyển sang biểu diễn bằng đồ thị; các chiến lược tìm kiếm mù và sau cùng là các kỹ thuật tìm kiếm theo kinh nghiệm được áp dụng cho bài toán có độ phức tạp tính toán cấp hàm mũ.

2.1. ĐẶT VẤN ĐỀ

Khi giải quyết bài toán bằng phương pháp tìm kiếm, trước hết ta phải xác định được *không gian tìm kiếm* (bao gồm tất cả các đối tượng mà trên đó thực hiện việc tìm kiếm). Nó có thể là không gian liên tục và nó cũng có thể là không gian các đối tượng rời rạc. Như vậy, ta sẽ xét việc biểu diễn một bài toán trong không gian trạng thái sao cho việc giải quyết bài toán này được quy về việc tìm kiếm lời giải trong không gian trạng thái. Ta có thể sử dụng các khái niệm *trạng thái* (state) và *toán tử* (operator) để biểu diễn bài toán đang xét.

Phương pháp giải quyết vấn đề dựa trên khái niệm trạng thái và toán tử được gọi là cách tiếp cận giải quyết vấn đề nhờ không gian trạng thái.

2.2. MỘT SỐ KHÁI NIỆM

2.2.1. Mô tả trạng thái

Giải bài toán trong không gian trạng thái, trước hết phải xác định dạng mô tả trạng thái bài toán sao cho bài toán trở nên đơn giản hơn, phù hợp bản chất vật lý của bài toán (Có thể sử dụng các dấu ký hiệu, vectơ, mảng hai chiều, cây, danh sách,...).

Mỗi *trạng thái* chính là mỗi *hình trạng* của bài toán, các tình trạng ban đầu và tình trạng cuối của bài toán gọi là trạng thái đầu và trạng thái cuối.

Ví dụ 2.1: Bài toán trò chơi 8 số

Trong bảng ô vuông 3 hàng, 3 cột, mỗi ô chứa một số nằm trong phạm vi từ 1 đến 8 sao cho không có 2 ô có cùng giá trị, có một ô trong bảng bị trống (không chứa giá trị nào cả). Xuất phát từ một sắp xếp nào đó các số trong bảng, hãy dịch chuyển ô trống sang phải, sang trái, lên trên hoặc xuống dưới (nếu có thể được) để đưa bảng ban đầu về bảng quy ước

2	8	3
1	6	4
7	■	5

Trạng thái đầu

trước.

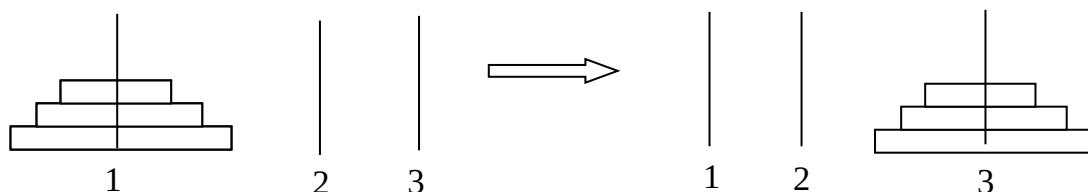
1	2	3
8	■	4
7	6	5

Trạng thái đích

Hình 2.1. Bài toán 8 số

Mỗi hình trạng trong bài toán này là một cách sắp xếp các con số. Ta có thể dùng mảng hai chiều kích thước 3×3 hoặc mảng một chiều kích thước 9 để biểu diễn cho mỗi trạng thái trong máy tính.

Ví dụ 2.2: Bài toán tháp Hà nội



Hình 2.2. Bài toán tháp Hà Nội với $n = 3$

Cho 3 cọc 1, 2, 3. Ở cọc 1 ban đầu có n đĩa sắp xếp theo thứ tự từ nhỏ đến lớn. Hãy dịch chuyển n đĩa đó sang cọc 3 sao cho:

- Mỗi lần chuyển một đĩa.
- Trong mỗi cọc không được đặt đĩa to ở trên đĩa nhỏ hơn trong bất cứ tình huống nào.

Trong bài toán trên, mỗi trạng thái là một bộ (ijk) với ý nghĩa: Đĩa C (đĩa lớn nhất) ở cọc i , đĩa B ở cọc j , đĩa A (đĩa bé nhất) ở cọc k . Trạng thái đầu là (111) còn trạng thái đích là (333) .

Với bài toán này ta cũng có thể dùng mảng hai chiều để biểu diễn: mảng này gồm 3 cột tương ứng với 3 cọc, số dòng tương ứng với số đĩa. Cụ thể trong trường hợp này ta dùng mảng 3×3 .

A		
B		
C		

Trạng thái đầu

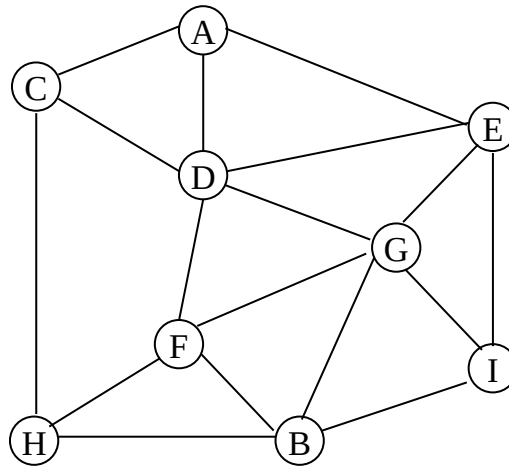
		A
		B
		C

Trạng thái đích

Hình 2.3. Trạng thái trong bài toán tháp Hà Nội

Ví dụ 2.3: Bài toán khách du lịch

Một khách du lịch có trong tay bản đồ mạng lưới giao thông nối các thành phố trong một vùng lãnh thổ (hình 2.4). Du khách đang ở thành phố A và anh ta muốn tìm đường đi tới thành phố B.



Hình 2.4. Tìm đường đi từ A đến B

Trong bài toán này, các thành phố có trong bản đồ là các trạng thái. Thành phố A là trạng thái đầu còn B là trạng thái kết thúc. Với bài toán này ta có thể sử dụng cách biểu diễn của đồ thị (ma trận kề, ma trận trọng số, danh sách cạnh, danh sách kề).

2.2.2. Toán tử

Toán tử là các phép biến đổi từ trạng thái này sang trạng thái khác.

Có hai cách dùng để biểu diễn các toán tử:

- Biểu diễn như một hàm xác định trên tập các trạng thái và nhận giá trị cũng trong tập này.
- Biểu diễn dưới dạng các quy tắc sản xuất $S \rightarrow A$ có nghĩa là nếu có trạng thái S thì có thể đưa đến trạng thái A.

Trong ví dụ 2.1, mỗi toán tử là cách chuyển ô trống, có 4 kiểu toán tử : chuyển ô trống lên trên, xuống dưới, sang trái, sang phải. Tuy nhiên đối với một số trạng thái nào đó, một toán tử nào đó có thể không áp dụng được, chẳng hạn nếu ô trống nằm ở cột đầu tiên thì ô trống không thể sang trái được.

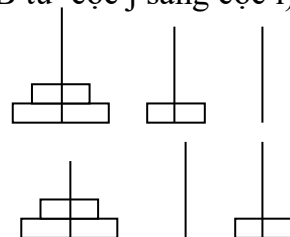
Trong ví dụ 2.2, toán tử là cách chuyển đĩa từ cọc này sang cọc khác, mỗi lần chuyển một đĩa và không được đặt đĩa to ở trên đĩa nhỏ.

Chẳng hạn như: $(ijk) \rightarrow (ijj)$ (Chuyển đĩa A từ cọc k sang cọc j)

$(ijk) \rightarrow (iik)$ (Chuyển đĩa B từ cọc j sang cọc i)

$(1\ 1\ 1) \rightarrow (1\ 1\ 2)$

$(1\ 1\ 1) \rightarrow (1\ 1\ 3)$



Trong ví dụ 2.3, mỗi toán tử là hành động đi từ thành phố này tới các thành phố khác.

2.3. CẤU TRÚC CHUNG CỦA BÀI TOÁN TÌM KIẾM

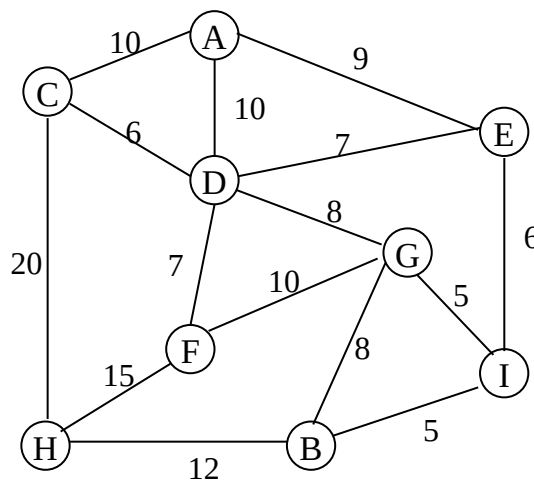
Từ các ví dụ 2.1, 2.2, 2.3 ta có thể thấy nhiều bài toán đều có dạng "tìm đường đi trong đồ thị", hay nói một cách hình thức hơn là "*xuất phát từ một đỉnh của một đồ thị, tìm đường đi hiệu quả nhất đến một đỉnh nào đó*". Từ đây ta có bài toán phát biểu trong không gian trạng thái.

Bài toán S: Cho trước hai trạng thái T_0 và T_G hãy xây dựng chuỗi trạng thái $T_0, T_1, T_2, \dots, T_{n-1}, T_n = T_G$ sao cho:

$$\sum_{i=1}^n \text{cost}(T_{i-1}, T_i) \text{ thỏa mãn một điều kiện cho trước (thường là nhỏ nhất).}$$

Trong đó, T_i thuộc tập hợp S (gọi là không gian trạng thái – state space) bao gồm tất cả các trạng thái có thể có của bài toán, và $\text{cost}(T_{i-1}, T_i)$ là *chi phí để biến đổi* từ trạng thái T_{i-1} sang trạng thái T_i . Dĩ nhiên, từ một trạng thái T_{i-1} ta có nhiều cách để biến đổi sang trạng thái T_i . Khi nói đến một biến đổi cụ thể từ T_{i-1} sang T_i ta sẽ dùng thuật ngữ *hướng đi* (với ngụ ý nói về sự lựa chọn).

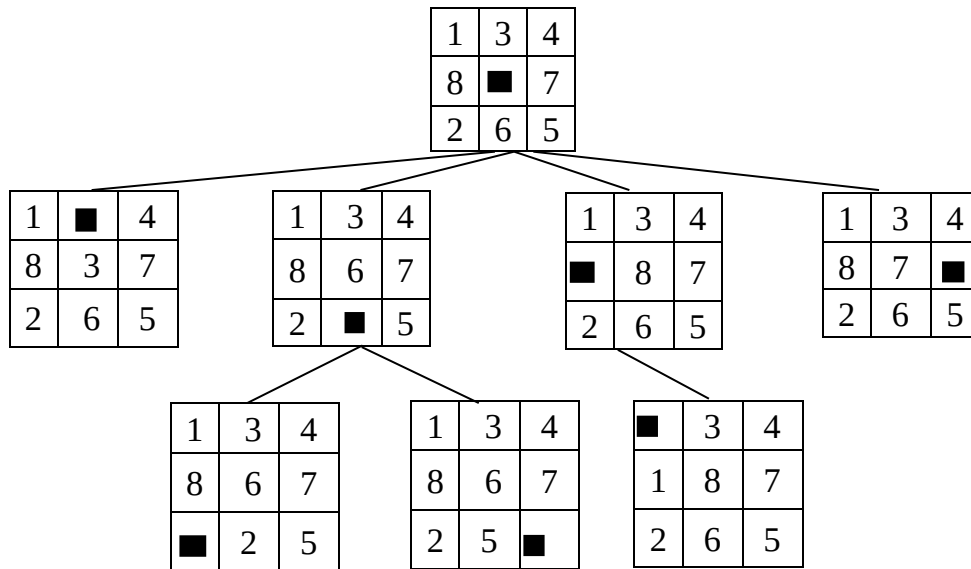
Mô hình chung của các bài toán phải giải quyết bằng phương pháp tìm kiếm lời giải. Không gian tìm kiếm là một tập hợp trạng thái - tập các nút của đồ thị. Chi phí cần thiết để chuyển từ trạng thái T_{i-1} này sang trạng thái T_i được biểu diễn dưới dạng các con số nằm trên cung nối giữa hai nút.



Hình 2.5. Đồ thị có trọng số

Chẳng hạn, với đồ thị trong hình 2.5 ta có thể phát biểu bài toán: Hãy tìm đường đi ngắn nhất từ đỉnh A đến đỉnh B.

Đa số các bài toán thuộc dạng mà chúng ta đang mô tả đều có thể được biểu diễn dưới dạng đồ thị. Trong đó, mỗi trạng thái là một đỉnh của đồ thị. Tập hợp S bao gồm tất cả các trạng thái chính là tập hợp bao gồm tất cả các đỉnh của đồ thị. Việc biến đổi từ trạng thái T_{i-1} sang trạng thái T_i là việc đi từ đỉnh đại diện cho T_{i-1} sang đỉnh đại diện cho T_i theo cung nối giữa hai đỉnh này.



Hình 2.6. Một phần đồ thị biểu diễn trò chơi 8 số

Bài toán G: Cho đỉnh đầu S_0 và tập các đỉnh đích Goal. Hãy tìm đường đi p (tối ưu) nào đó từ đỉnh S_0 đến đỉnh nào đó thuộc tập Goal.

Từ những phân tích ở trên ta có thể thấy được sự tương đương giữa không gian trạng thái và đồ thị được tổng hợp ở bảng sau:

Không gian trạng thái	Đồ thị
- Trạng thái đầu - Trạng thái đích - Toán tử dịch chuyển - Dãy các trạng thái - Bài toán S	- Đỉnh đầu - Đỉnh đích - Cung - Đường đi - Bài toán G

2.4. TÌM KIẾM LỜI GIẢI TRONG KHÔNG GIAN TRẠNG THÁI

Quá trình tìm kiếm lời giải của bài toán được biểu diễn trong không gian trạng thái được xem như quá trình dò tìm trên đồ thị, xuất phát từ trạng thái ban đầu, thông qua các toán tử chuyển trạng thái, lần lượt đến các trạng thái tiếp theo cho đến khi gặp được trạng thái đích hoặc không còn trạng thái nào có thể tiếp tục được nữa.

Khi áp dụng các phương pháp tìm kiếm trong không gian trạng thái, người ta thường quan tâm đến các vấn đề sau:

- Kỹ thuật tìm kiếm lời giải
- Phương pháp luận của việc tìm kiếm
- Chiến lược tìm kiếm

Tuy nhiên, không phải các phương pháp này đều có thể áp dụng để giải quyết cho tất cả các bài toán phức tạp mà chỉ cho từng lớp bài toán.

Việc chọn chiến lược tìm kiếm cho bài toán cụ thể phụ thuộc nhiều vào các đặc trưng của bài toán.

Trong phần này, chúng ta sẽ nghiên cứu hai chiến lược tìm kiếm:

- Các kỹ thuật tìm kiếm *mù*: Trong các kỹ thuật tìm kiếm này, không có một sự hướng dẫn nào cho sự tìm kiếm, mà ta chỉ phát triển các trạng thái một cách hệ thống từ trạng thái ban đầu cho tới khi gặp một trạng thái đích nào đó. Có ba kỹ thuật tìm kiếm mù cơ bản, đó là tìm kiếm theo chiều rộng (Breadth First Search), tìm kiếm theo chiều sâu (Depth First Search) và tìm kiếm sâu dần.

- Các kỹ thuật tìm kiếm *kinh nghiệm* (tìm kiếm heuristic): với kỹ thuật tìm kiếm này, phải dựa vào kinh nghiệm và sự hiểu biết của chúng ta về vấn đề cần giải quyết để xây dựng nên hàm đánh giá nhằm tìm ra các đỉnh tiềm năng dẫn đến lời giải. Trong số các trạng thái chờ phát triển, ta chọn trạng thái được đánh giá là tốt nhất để phát triển. Do đó tốc độ tìm kiếm sẽ nhanh hơn.

Nhận xét:

- i) Trong tìm kiếm mù: ta chọn trạng thái để phát triển theo thứ tự mà chúng được sinh ra.
- ii) Trong tìm kiếm kinh nghiệm: ta chọn trạng thái để phát triển dựa vào sự đánh giá các trạng thái.